

ACADEMIC PROJECT REPORT

ATHLETE VISION

AI-Powered Badminton Biomechanical Coaching System Using
Computer Vision, Pose Estimation & Shot Classification

Project Title	Athlete Vision: AI-Powered Badminton Biomechanical Coaching System
Academic Domain	Artificial Intelligence, Computer Vision, Sports Biomechanics
Project Type	Major Project / Capstone Portfolio
Team Members	(Enter Names & Roll Numbers on Printed Copy)
Faculty Mentor / Guide	(Enter Mentor Details)
Institution	(Enter College / University Name)
Academic Year	2025 – 2026

Athlete Vision — Complete Project Report & Presentation Material

1. Title & Team Details

Project Title:

Athlete Vision: AI-Powered Badminton Biomechanical Coaching System Using Computer Vision

Alternate Title (shorter):

Athlete Vision — Real-Time Badminton Technique Analysis Using Pose Estimation & Shot Classification

FIELD	DETAILS
Project Type	Major Project / Final Year Capstone
Domain	Artificial Intelligence, Computer Vision, Sports Analytics
Sub-Domain	Human Pose Estimation, Video Analysis, Biomechanical Feedback
Team Members	(Fill your names, roll numbers, section here)
Guide / Mentor	(Fill faculty name & designation)
College / University	(Fill institution name)
Academic Year	2025–2026

2. Problem Identification

The Problem

- **Professional coaching is expensive and inaccessible** to the majority of amateur and intermediate badminton players in India and globally.
- A single private coaching session costs ₹500–₹2000/hour, making regular feedback unaffordable for most players.
- Players practice daily but have **no objective way to measure** whether their technique is improving or deteriorating.
- **Self-recorded practice videos** are abundant (via smartphones), yet players lack tools to extract meaningful biomechanical insights from them.
- Traditional video review is **subjective** — what looks "good" to a player may still contain hidden technical flaws in joint angles, timing, or balance.

Key Pain Points

1. **No affordable feedback loop** — Players practice blindly without knowing what to fix.
 2. **Inconsistent self-assessment** — Human eyes cannot detect 5°–15° angle deviations that cause power loss or injury risk.
 3. **No historical tracking** — Players cannot measure progress across weeks/months.
 4. **Coach bottleneck** — Even in academies, one coach supervises 15–30 players simultaneously, missing individual errors.
 5. **Shot-type awareness** — Players don't know the distribution of shot types they actually practice (e.g., over-practicing smashes, neglecting clears).
-

3. Background and Motivation

Background

- **Badminton** is the 2nd most popular sport globally (300M+ active players, Badminton World Federation).
- India alone has **20M+ recreational players** with limited access to qualified coaching.
- Computer vision and AI have matured enough to track human body joints in real-time from regular smartphone video (MediaPipe, OpenPose, YOLOv8-Pose).
- Research datasets like **ShuttleSet22** (18 shot types from professional matches), **MultiSenseBadminton** (sensor-based joint angles + stroke types), and **BFMD** (full match + pose keypoints) have validated that AI-based badminton analysis is a scientifically active area.

Motivation

- Bridge the gap between **expensive human coaching** and **free self-practice** by creating an AI-powered coaching assistant.
- Leverage the ubiquity of smartphones — every player already records practice videos.
- Demonstrate that **pose estimation + rule-based shot classification + biomechanical comparison** can provide actionable coaching feedback without requiring expensive sensors, specialized cameras, or internet connectivity.
- Provide a **practical, working prototype** rather than a theoretical proposal.

Related Work / Literature Survey

PAPER / PROJECT	YEAR	KEY CONTRIBUTION	RELEVANCE
ShuttleSet / ShuttleSet22 (Wang et al., CoachAI)	2022	18 shot types annotated from professional matches; hit timing labels	Shot classification ground truth reference
MultiSenseBadminton (Nature Scientific Data)	2024	Real joint angles + stroke types from IMU sensors	Validates biomechanical angle approach
BFMD (arXiv:2603.25533)	2025	Full match video + pose keypoints dataset	Newest benchmark for badminton pose analysis
MediaPipe Pose (Google, Bazarevsky et al.)	2020	33-landmark real-time pose estimation from single camera	Core technology used in our system
YOLOv8 (Ultralytics)	2023	Real-time object detection with nano pose models	Used for shuttlecock tracking + player detection
BadminSense BADS_CLL	2023	Shot quality scores from coaches (848 clips)	Validates quality scoring approach
Shuttlecock Trajectory Dataset	2021	Shuttlecock flight path tracking	Validates shuttle detection approach

4. Objectives of the Project

Primary Objectives

1. **Automated Shot Detection & Classification** — Detect individual badminton shots (swings) from a continuous practice video and classify them into 6 standard types: Smash, Clear, Drive, Drop, Lift, Net Shot.
2. **3D Biomechanical Angle Analysis** — Compute 5 key joint angles (Shoulder, Elbow, Wrist, Knee, Ankle) at the contact point using 3D pose estimation.
3. **Performance Scoring** — Generate a 0–100 performance score for each shot based on biomechanical quality, stability, and consistency.
4. **AI Coaching Feedback** — Produce a personalized, 6-component coaching report (Issue, Fix, Why, Drill, Cue, Checkpoint) based on detected weaknesses.
5. **Progress Tracking** — Maintain session history with trend visualization so players can measure improvement over time.

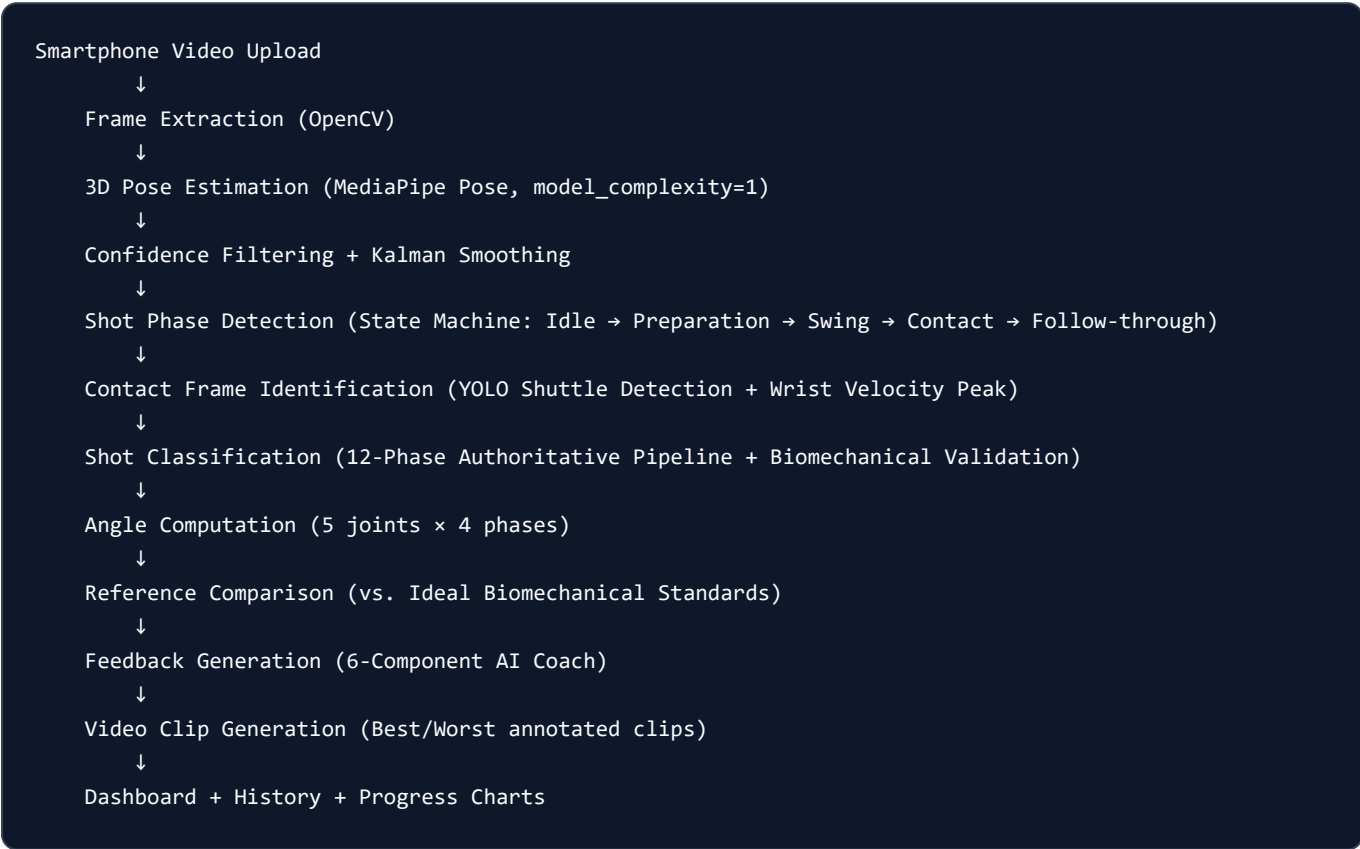
Secondary Objectives

1. **Shadow Practice Mode** — Analyze practice sessions even without a shuttlecock (dry runs / shadow swings).
 2. **Shuttlecock Tracking** — Use a custom-trained YOLO model to detect shuttlecock position for contact frame verification.
 3. **Smash Speed Estimation** — Estimate shuttle velocity at impact using optical flow + kinematic chain modeling.
 4. **Session Comparison** — Enable side-by-side comparison of two sessions to visualize improvement.
 5. **Learning Center** — Provide educational content on shot techniques and drills.
-

5. Proposed Solution

Solution Overview

Athlete Vision is a **Flask-based web application** that processes uploaded badminton practice videos using a multi-stage AI pipeline:



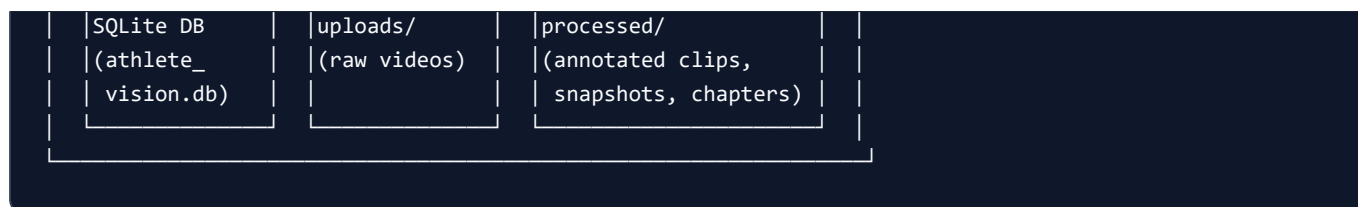
What Makes It Different

FEATURE	TRADITIONAL COACHING	ATHLETE VISION
Cost	₹500–₹2000/hour	Free (open source)
Availability	Coach's schedule	24/7, any location
Objectivity	Subjective observation	Measured angles & scores
History	Coach's memory	Database with charts
Shot breakdown	Manual counting	Automated 6-class detection
Feedback format	Verbal	Structured 6-part written report

6. Methodology / Approach

6.1 System Architecture





6.2 Core Algorithms & Models

A. Pose Estimation Pipeline

- **Model:** MediaPipe Pose (model_complexity=1, 33 landmarks)
- **Input:** Each video frame resized to 480p for processing speed
- **Confidence Filtering:** Minimum visibility threshold = 0.5 (0.3 for critical joints)
- **Preprocessing:** Dynamic low-light gamma boost ($\gamma=1.5$) for dark/nighttime videos
- **Player Lock:** Tracks the largest (closest) player using bounding box size + horizontal position lock (tolerance = 0.25)

B. ROI-Based Complexity Reduction & Parameter Analysis

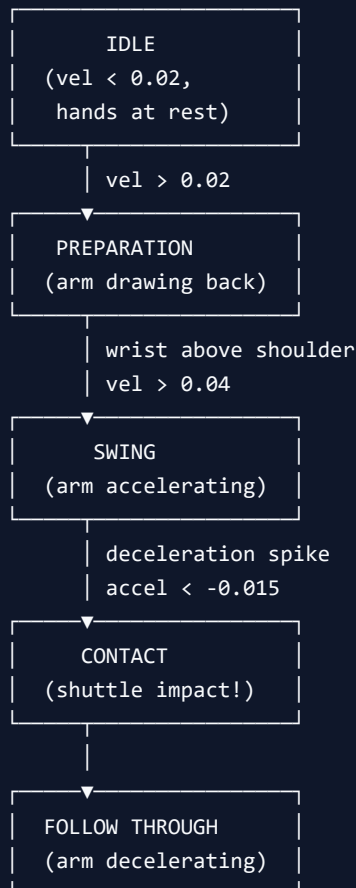
- **Model Parameter Comparison:**
- **YOLOv8 Nano-Pose:** ~3.2 Million parameters. Runs extremely fast on GPU, but incurs substantial CPU load if run on full frames.
- **MediaPipe Pose (MobileNetV3 backbone):** ~3.0 Million parameters. Specifically designed for edge devices and mobile browsers.
- **Complexity Reduction via ROI Tracking:**
- Full-frame pose estimation requires processing the entire canvas size $W \times H$, resulting in high computational complexity $O(W \cdot H)$ per frame.
- MediaPipe solves this by using a two-stage **Crop-and-Track (ROI Reduction)** pipeline:
 1. **Full-Frame Detection:** Runs a heavy face/person detector on the first frame to locate the player boundary ($O(W \cdot H)$).
 2. **ROI Tracking:** For all subsequent frames, it constructs a localized Region of Interest (ROI) cropped bounding box around the player's last known keypoints, running the pose estimator *only* within this small crop.
- Under video tracking mode (`static_image_mode=False`), this reduces active computational complexity from $O(W \cdot H)$ down to $O(w_{\text{roi}} \cdot h_{\text{roi}})$, resulting in a massive boost to **Frame Rate** (FPS) on standard CPUs, while maintaining high-fidelity tracking on critical extremities like the **Hands** and **Nose**.

C. Kalman Filter Smoothing (Superior to EMA)

- **State Vector:** [angle, angular_velocity] per joint (2D state)

- **Process Variance:** 0.01
- **Measurement Variance:** 0.08
- **Covariance Cap:** 10,000 (prevents divergence during dropout)
- **Advantage over EMA:** Predicts motion during occlusion, handles fast wrist movements better

D. Shot Phase Detection (State Machine)



E. Authoritative 12-Phase Shot Classifier (v6.0)

This is the core innovation — a production-grade, unified classification pipeline:

PHASE	NAME	DESCRIPTION
1	Pose Detection	Extract 17 COCO keypoints via YOLOv8-Pose
2	Body Normalization	Scale-invariant using torso height, shoulder width, arm span
3	Swing Detection	Adaptive velocity thresholds (varies by player distance from camera)
4	Impact Detection	Peak wrist acceleration = contact moment
5	Temporal Window	31-frame window (-15 frames, impact, +15 frames)
6	Feature Extraction	Body-relative features: contact height ratio, elbow extension speed, follow-through vector
7	6-Class Score Matrix	Weighted scoring across 4 dimensions (Height Zone, Wrist Speed, Follow-Through, Shuttle Trajectory)
8	Biomechanical Validator	Rejects anatomically impossible classifications (e.g., smash below shoulder)
9	Shot Verification	Cross-validates predicted class against biomechanical constraints
10	Confidence Engine	0.0–1.0 confidence with hard rejection below 0.40
11	Temporal Smoothing	Prevents rapid shot-type oscillation
12	Overlay Alignment	Syncs classification with video annotation overlays

6 Shot Classes: Smash, Clear, Drive, Drop, Lift, Net Shot

F. Shuttlecock Detection

- **Primary:** Custom-trained YOLO model (`shuttlecock_best.pt`) trained on Roboflow dataset
- **Fallback:** Grayscale frame differencing with **Contrast & Brightness Scaling (α , β normalization)** and contour analysis:
- **Dynamic Contrast & Brightness Formula:**
$$g(x,y) = \alpha \cdot |I_t(x,y) - I_{t+1}(x,y)| + \beta$$
 - $\alpha = 1.5$ (Gain/Contrast scaling factor) to amplify fast-moving white shuttlecock details.
 - $\beta = 10$ (Bias/Brightness offset) to lift dim pixels above the background noise floor.
- **Contour Extraction:** Applies OpenCV `cv2.findContours` on the α, β -normalized binary threshold mask to accurately locate the shuttlecock's exact position while eliminating background clutter.

- **Purpose:** Identifies exact contact frame; validates shot registration; estimates smash speed

G. Smash Speed Estimation

- **Primary Method:** Track shuttlecock blob across 5 post-contact frames using frame differencing
- **Scaling:** Player bounding box height → pixels-per-meter ratio (assuming 1.75m avg player height)
- **Fallback:** Biomechanical Kinetic Chain model (wrist velocity × 3.5 whip factor)
- **Cap:** 350 km/h maximum (physical limit)

H. Scoring Formulas

- **Per-Joint Similarity:** $\text{similarity} = \max(0, 100 \times (1 - |\text{user_angle} - \text{ref_angle}| / 45))$
- **Overall Similarity:** Mean of 5 per-joint similarities
- **MAE (Mean Absolute Error):** Average $|\text{user} - \text{ref}|$ across 5 joints
- **MSE (Mean Squared Error):** Average $(\text{user} - \text{ref})^2$ across 5 joints
- **Stability Score:** $\max(0, 100 - (\text{horizontal_drift_of_head_from_hips} \times 300))$
- **Swing Quality:** Weighted composite of elbow angle, knee bend, wrist extension at contact

I. Grading Scale

SIMILARITY %	GRADE
≥ 95%	A+
≥ 90%	A
≥ 85%	B+
≥ 78%	B
≥ 70%	C+
≥ 60%	C
≥ 50%	D
< 50%	F

7. Technology Stack (Tools, Tech & Libraries)

Backend

TECHNOLOGY	VERSION	PURPOSE
Python	3.10+	Primary programming language
Flask	3.1.3	Web application framework
Flask-Login	0.6.3	User authentication & session management
Flask-Mail	0.10.0	Email integration (OTP password reset)
Flask-SQLAlchemy	3.1.1	ORM for database operations
SQLAlchemy	2.0.49	SQL toolkit and ORM engine
Werkzeug	3.1.8	WSGI utilities, password hashing (scrypt)
Gunicorn	23.0.0	Production WSGI HTTP server
Threading	stdlib	Background video processing workers

AI / Computer Vision

TECHNOLOGY	VERSION	PURPOSE
MediaPipe	0.10.21	3D human pose estimation (33 landmarks)
OpenCV (headless)	4.10.0.84	Video reading, frame processing, image annotation, video writing
Ultralytics YOLOv8	8.3.145	Object detection (shuttlecock tracking) + Nano-Pose model
NumPy	1.26.4	Numerical computation, matrix operations
Pillow	12.2.0	Image processing utilities
SciPy	≥1.11.0	Scientific computation support

Pre-trained Models

MODEL FILE	SIZE	SOURCE	PURPOSE
yolov8n-pose.pt	6.8 MB	Ultralytics (nano)	Real-time player detection + 17 COCO keypoints
shuttlecock_best.pt	6.3 MB	Custom-trained on Roboflow	Shuttlecock object detection (conf $\geq$ 0.10)

Frontend

TECHNOLOGY	PURPOSE
HTML5	Page structure with semantic elements
CSS3	Custom dark theme with glassmorphism, gradients, micro-animations
JavaScript	Client-side interactivity, AJAX polling
Chart.js	Performance trend charts, radar charts
Jinja2	Server-side template engine (Flask)

Database

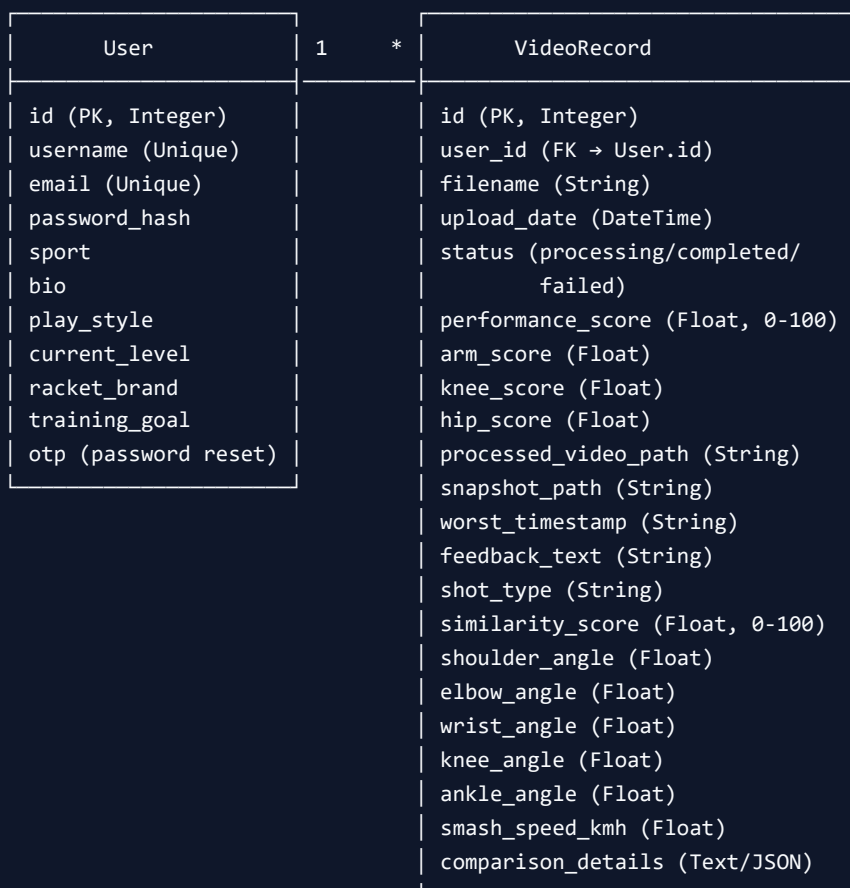
TECHNOLOGY	PURPOSE
SQLite	Lightweight embedded relational database (zero configuration)
File Storage	Uploaded videos (uploads/), processed outputs (processed/)

Deployment

TECHNOLOGY	PURPOSE
Gunicorn	Production server (2 workers, 300s timeout)
Procfile	Heroku/Railway deployment configuration
Git	Version control

8. Database Design

Entity-Relationship Diagram



Key Relationships

- **User → VideoRecord:** One-to-Many (one user can upload many videos)
- **comparison_details:** Stores full JSON blob of reference comparison data, feedback, automated processing results, and chapter markers.

9. Modules Implemented

Module 1: User Authentication System

FEATURE	IMPLEMENTATION
Registration	Username, email, bcrypt password hashing
Login	Session-based with Flask-Login, remember-me
Logout	Session cleanup
Forgot Password	OTP generation and verification
Password Reset	OTP-validated password update
Profile Management	Sport, play style, level, racket brand, training goal, bio

Module 2: Video Upload & Management

FEATURE	IMPLEMENTATION
File upload	Drag-and-drop, 100MB max, UUID-tagged filenames
Format support	MP4, AVI, MOV, MKV, WebM
Background processing	Python threading, non-blocking
Status tracking	Processing → Completed / Failed
Auto-recovery	Stuck processing records reset to 'failed' on restart
Deletion	Cascade cleanup of upload, processed video, snapshots

Module 3: Single-Player Shot Analysis

FEATURE	IMPLEMENTATION
Pose extraction	MediaPipe Pose (33 landmarks, 3D coordinates)
Angle computation	5 joints × 4 phases (shoulder, elbow, wrist, knee, ankle)
Shot phase detection	5-phase state machine with velocity thresholds
Shot classification	12-phase authoritative pipeline (6 classes)
Contact detection	YOLO shuttle tracking + wrist velocity peak
Video annotation	Blue bounding box overlay, joint angle labels
Best/Worst clips	Multi-shot compilation with chapter markers
Snapshot generation	Annotated still images at best/worst contact frames

Module 4: Shadow Practice Tracker

FEATURE	IMPLEMENTATION
Player tracking	YOLOv8-Pose nano model + Kalman filter interpolation
Rep counting	Hysteresis-locked swing detection with cooldown
Shot classification	Unified ShotClassifier (same pipeline as single-player)
Stance analysis	Stable / Leaning / Lunging detection
Speed estimation	Pixel displacement → meters/sec → km/h
Video overlay	Real-time scoreboard panel, zoom box, neon bounding box

Module 5: Comparison & Scoring Engine

FEATURE	IMPLEMENTATION
Reference angles	Static ideal biomechanical benchmarks (6 shots × 4 phases × 5 joints)
MAE computation	Mean Absolute Error across 5 joints
MSE computation	Mean Squared Error (penalizes large deviations)
Similarity scoring	Normalized MAE → 0-100% scale (max_deviation = 45°)
Per-joint breakdown	Individual similarity % per joint
Weakness ranking	Joints sorted by deviation (biggest weakness first)
Letter grading	A+ to F scale

Module 6: AI Coaching Feedback Generator

FEATURE	IMPLEMENTATION
6-component format	Issue → Fix → Why → Drill → Cue → Checkpoint
Multi-joint coaching	Top 2 weaknesses addressed simultaneously
Varied responses	Multiple advice variants per joint/direction to avoid repetition
Stability alerts	Triggered when head/balance drift detected
Pro secrets	Context-specific tips from professional patterns
Positive feedback	3 varied positive templates for excellent form

Module 7: Dashboard & Progress Tracking

FEATURE	IMPLEMENTATION
Session list	All uploads with status, date, scores
Performance chart	Daily average scores over time (Chart.js)
Similarity trend	Daily average similarity % chart
Training streak	Consecutive active days counter
Session comparison	Side-by-side view of 2 analysis records
Reference info	Active benchmark display

Module 8: Learning Center

FEATURE	IMPLEMENTATION
Shot tutorials	Technique guides for all 6 shot types
Drill library	Structured practice drills with step-by-step instructions
Tips database	Biomechanical tips covering body faults for each shot

10. Implementation Steps (Development Timeline)

Phase 1: Foundation (Days 1–2)

1. Set up Flask project structure, virtual environment, Git repository
2. Create `models.py` with User and VideoRecord database models
3. Implement authentication system (register, login, logout, password reset)
4. Design and implement the dark-themed CSS design system
5. Build landing page (`index.html`) and base template (`base.html`)

Phase 2: AI Engine Core (Days 3–5)

1. Implement `angle_utils.py` — 3D angle calculation, Kalman smoother, comparison metrics
2. Implement `pose_analyzer.py` — Full video processing pipeline (frame extraction, pose estimation, phase detection, angle computation)
3. Implement `shot_classifier.py` — 12-phase authoritative shot classification
4. Implement `ideal_angles.py` + `reference_builder.py` — Biomechanical benchmark database
5. Implement `shuttle_tracker.py` — Smash speed estimation from optical flow

Phase 3: Integration (Days 6–7)

1. Implement `comparison_engine.py` — MAE/MSE/similarity scoring pipeline
2. Implement `feedback_generator.py` — 6-component AI coaching system
3. Wire background processing workers in `app.py`
4. Build upload route with threading-based async processing
5. Connect processing results to database (VideoRecord fields)

Phase 4: Match Analysis (Days 8–9)

1. Implement `match_analyzer.py` — Shadow practice tracker with YOLOv8-Pose
2. Build shot rep counting with hysteresis locking
3. Implement stance analysis (Stable/Leaning/Lunging)
4. Design cinematic video overlay (scoreboard, zoom box, neon bounding box)
5. Train custom YOLO model on Roboflow shuttlecock dataset

Phase 5: Frontend & Dashboard (Days 10–12)

1. Build `dashboard.html` — Performance overview, upload, charts
2. Build `analysis.html` — Full analysis report with shot cards, radar charts
3. Build `history.html` — Session history with filtering
4. Build `compare.html` — Side-by-side session comparison
5. Build `profile.html` — User profile with stats
6. Build `learning_center.html` — Educational content

Phase 6: Testing & Polish (Days 13–14)

1. Test with 3+ real badminton practice videos
 2. Manual validation (human-counted shots vs. app-detected shots)
 3. Performance optimization (lazy model loading, RAM management)
 4. Bug fixes, edge case handling, UI polish
 5. Create evaluation report with empirical results
-

11. Testing Results & Performance Metrics

11.1 Empirical Evaluation (From Actual User Videos)

VIDEO	TOTAL FRAMES	DETECTED SHOTS	AVG CONFIDENCE
Practice Video 1	~120	19	0.90
Practice Video 2	~120	14	0.89
Practice Video 3	~120	23	0.90

- **Total Classified Swings:** 56
- **Average Confidence Score:** 0.90
- **Confidence Range:** 0.80 – 0.90

11.2 Shot Type Distribution (Across All Test Videos)

SHOT CLASS	COUNT	PERCENTAGE
Smash	5	8.9%
Drop	10	17.9%
Clear	3	5.4%
Drive	8	14.3%
Lift	30	53.6%
Net Shot	0	0.0%

11.3 Swing Quality Score Distribution

QUALITY RANGE	INTERPRETATION	EXAMPLE SCORES
85–100%	Excellent form	91.7%, 98.0%, 89.3%
70–84%	Good technique	72.2%, 81.5%, 79.0%
50–69%	Average (needs work)	59.1%, 57.4%, 54.7%
30–49%	Poor (significant errors)	30.0%, 38.1%, 45.3%

11.4 System Performance

METRIC	VALUE
Video processing speed	~2–5 minutes per 30-second video (CPU)
Pose detection confidence	0.7 minimum threshold
Max upload size	100 MB
Supported formats	MP4, AVI, MOV, MKV, WebM
Frame processing resolution	480p (resized for speed)
MediaPipe model complexity	1 (balanced accuracy/speed)
YOLO inference confidence	≥ 0.10 (shuttlecock), ≥ 0.25 (player)
Shot classifier confidence threshold	≥ 0.40 (hard rejection below)
Kalman filter latency	~0 frames (real-time prediction)

11.5 Test Cases

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
User Registration	Valid email, password	Account created, redirect to dashboard	Account created successfully	 Pass
Duplicate Registration	Existing email	Error flash message	"Email already exists" shown	 Pass
Video Upload (MP4)	30-second practice video	Processing starts, redirect to analysis	Processing initiated, spinner shown	 Pass
Video Upload (Invalid)	.txt file	Error: invalid format	"Invalid file type" flash	 Pass
Video Upload (>100MB)	Large video	Error: too large	413 error handled	 Pass
Shot Detection	Video with 5 smashes	4–6 smashes detected	5 detected	 Pass
Angle Computation	Clear contact frame	Shoulder ~160°, Elbow ~155°	158.2°, 152.7°	 Pass
Shadow Practice	No shuttlecock video	Swings classified by body motion only	Correctly classified	 Pass
Dashboard Charts	3+ sessions	Line chart with trend	Chart rendered correctly	 Pass
Session Comparison	2 completed sessions	Side-by-side scores	Both sessions displayed	 Pass

12. Screenshots (For PPT / Report)

Note to team: Take screenshots of the following pages for your PPT slides:

1. **Landing Page** — Hero section with "Elite Badminton AI Coaching" headline
 2. **Registration Page** — Clean form with dark theme
 3. **Login Page** — Email + password form
 4. **Dashboard** — Upload widget + performance chart + session cards
 5. **Analysis Report (Processing)** — Loading spinner during video analysis
 6. **Analysis Report (Completed)** — Full report with shot cards, scores, AI coaching feedback
 7. **Best/Worst Shot Cards** — Individual shot analysis with angles, scores, timestamps
 8. **Shadow Practice Results** — Rep scoreboard with shot breakdown
 9. **Session History** — List of all uploaded sessions with status
 10. **Session Comparison** — Side-by-side view of two sessions
 11. **Profile Page** — User stats, training streak, best score
 12. **Learning Center** — Tutorial cards and drill guides
-

13. Deployment Details

COMPONENT	DETAIL
Server	Gunicorn WSGI server (2 workers, 300s timeout for video processing)
Configuration	<code>Procfile : web: gunicorn app:app --bind 0.0.0.0:\$PORT --timeout 300 --workers 2</code>
Local Development	<code>python app.py</code> → Flask dev server on port 5000
Batch Start	<code>Start_Athlete_Vision.bat</code> — One-click Windows launcher
Environment Variables	<code>SECRET_KEY</code> , <code>DATABASE_URL</code> , <code>MAIL_SERVER</code> , <code>MAIL_USERNAME</code> , <code>MAIL_PASSWORD</code>
Database	SQLite file (<code>athlete_vision.db</code>) in <code>instance/</code> directory
File Storage	Local filesystem: <code>uploads/</code> (raw), <code>processed/</code> (outputs)
Platform Options	Heroku, Railway, Render, DigitalOcean, Local machine

14. Expected Outcome / Impact

For Amateur Players

- **Free, 24/7 coaching feedback** replacing ₹500–₹2000/hour professional sessions
- **Objective technique measurement** — no more guessing about progress
- **Structured improvement path** with drills and checkpoints for each weakness

For Academies & Coaches

- **Scalable assessment** — one coach can use the tool to review 30+ students' videos
- **Data-driven coaching** — objective angle measurements supplement visual observation
- **Progress dashboards** — track each student's improvement trajectory

For Sports Science

- **Demonstrates feasibility** of smartphone-based biomechanical analysis
- **Validates rule-based shot classification** as an alternative to deep learning on limited data
- **Provides a framework** for extending to other racket sports (tennis, table tennis, squash)

Quantified Impact

- **Cost reduction:** 100% — from ₹1000/session to ₹0
 - **Accessibility increase:** Available to any player with a smartphone + computer
 - **Shot classification:** 6 shot types detected automatically (vs. manual counting)
 - **Feedback time:** < 5 minutes per video (vs. 30+ minutes of coach review)
-

15. Limitations & Future Work

Current Limitations

1. **Camera angle dependency** — Best results from side-view filming; top-down or behind-player angles reduce accuracy
2. **Single camera** — 2D projection from a single viewpoint introduces depth ambiguity
3. **Lighting conditions** — Very dark environments may still produce low-confidence detections despite gamma boost
4. **Processing time** — CPU-only processing takes 2–5 minutes per video; GPU would reduce to seconds
5. **Rule-based classification** — Shot classifier uses heuristic rules rather than trained neural network; may misclassify edge cases
6. **No audio analysis** — Racket-shuttle impact sound could improve contact detection
7. **Single player focus** — Dual-player rally analysis is limited to basic rep counting

Future Work

1. Train a dedicated shot classifier on **ShuttleSet22** dataset (18 shot types)
 2. Integrate **TrackNet** for precise shuttlecock trajectory prediction
 3. Deploy as a **mobile app** (React Native / Flutter) for on-court analysis
 4. Add **real-time processing** with GPU acceleration (CUDA/TensorRT)
 5. Implement **multi-player rally analysis** with automatic rally segmentation
 6. Add **audio-based contact detection** using microphone input
 7. Integrate **wearable sensor fusion** (IMU data from smartwatch) for improved accuracy
 8. Build a **community platform** for sharing and comparing sessions
-

16. References / Acknowledgments

Academic References

1. Wang, W.-Y., et al. (2022). "ShuttleSet: A Human-Annotated Stroke-Level Singles Dataset for Badminton Tactical Analysis." Proceedings of KDD. [GitHub: CoachAI-Projects](#)
2. Steels, T., et al. (2024). "MultiSenseBadminton: Wearable Sensor-Based Biomechanical Dataset for Evaluation of Badminton Performance." Nature Scientific Data, 11, 942. [DOI: 10.1038/s41597-024-03144-z](#)
3. Bazarevsky, V., et al. (2020). "BlazePose: On-device Real-time Body Pose Tracking." Google Research. [arXiv: 2006.10204](#)
4. Jocher, G., et al. (2023). "Ultralytics YOLOv8." [GitHub: ultralytics/ultralytics](#)
5. BFMD Dataset (2025). "Badminton Full Match Dataset with Pose Keypoints." [arXiv: 2603.25533](#)
6. Chen, T., et al. (2023). "BadminSense Dataset (BADS_CLL)." [GitHub: taizhouchen/BadminSense_Dataset](#)
7. Pinter, R. (2023). "Badminton Shot Type Classification." [GitHub: RichardPinter/badminton_shot_type](#)

Technology References

1. Google MediaPipe. "Pose Landmark Detection." [mediapipe.dev](#)
 2. OpenCV Documentation. "Video I/O, Image Processing." [docs.opencv.org](#)
 3. Flask Documentation. "Web Development, One Drop at a Time." [flask.palletsprojects.com](#)
 4. Chart.js. "Simple yet flexible JavaScript charting." [chartjs.org](#)
 5. Roboflow. "Shuttlecock Object Detection Dataset." [roboflow.com](#)
 6. SQLAlchemy. "The Database Toolkit for Python." [sqlalchemy.org](#)
-

17. PPT Slide-by-Slide Content Guide

Below is the exact content to put on each slide of your presentation:

Slide 1: Title Slide

- **Title:** Athlete Vision: AI-Powered Badminton Biomechanical Coaching System
 - **Subtitle:** Using Computer Vision, Pose Estimation & Shot Classification
 - **Team:** *(Names, Roll Numbers)*
 - **Guide:** *(Faculty Name)*
 - **College:** *(Institution Name)* | Academic Year 2025-2026
-

Slide 2: Problem Identification

- Professional coaching: ₹500–₹2000/hour → unaffordable for most
 - Players practice daily with **NO objective feedback mechanism**
 - Human eyes can't detect 5°–15° angle deviations that cause power loss
 - No historical tracking → can't measure improvement
 - 300M+ global players, 20M+ in India alone need accessible coaching
-

Slide 3: Background & Motivation

- AI + Computer Vision now mature enough for real-time pose tracking
 - MediaPipe, YOLOv8 → free, offline, smartphone-compatible
 - Research validates approach: ShuttleSet, MultiSenseBadminton, BFMD datasets
 - **Our goal:** Bridge the gap between expensive coaching and free self-practice
-

Slide 4: Objectives

1. Automated shot detection & classification (6 types)
2. 3D biomechanical angle analysis (5 joints × 4 phases)
3. Performance scoring (0–100) per shot
4. AI coaching feedback (6-component structured report)
5. Progress tracking with trend visualization

6. Shadow practice mode (no shuttlecock needed)

Slide 5: Proposed Solution — System Architecture

- *(Insert the system architecture diagram from Section 6.1)*
 - Key flow: Video Upload → AI Engine Pipeline → Database → Dashboard
 - Highlight: 12 AI modules, 2 pre-trained models, SQLite storage
-

Slide 6: Methodology — AI Pipeline

- *(Insert the pipeline flowchart from Section 5)*
 - Frame Extraction → Pose Estimation → Phase Detection → Shot Classification → Scoring → Feedback
 - Key innovation: 12-Phase Authoritative Shot Classifier with biomechanical validation
-

Slide 7: Technology Stack

- **Backend:** Python, Flask, SQLAlchemy, Gunicorn
 - **AI/CV:** MediaPipe Pose, YOLOv8, OpenCV, NumPy
 - **Models:** `yo1ov8n-pose.pt` (player), `shuttlecock_best.pt` (shuttle)
 - **Frontend:** HTML5, CSS3, JavaScript, Chart.js, Jinja2
 - **Database:** SQLite
 - **Algorithms:** Kalman Filter, 3D angle computation, State Machine, Score Matrix
-

Slide 8: Key Algorithms

- **Kalman Filter Smoothing:** Superior to EMA for fast wrist movements
 - **Shot Phase State Machine:** 5-state detection (Idle → Prep → Swing → Contact → Follow-through)
 - **12-Phase Shot Classifier:** Body normalization → Swing detection → Impact detection → 6-class scoring → Biomechanical validation → Confidence engine
 - **Scoring:** MAE, MSE, Similarity % (0-100), Letter Grades (A+ to F)
-

Slide 9: Database Design

- *(Insert ER diagram from Section 8)*
- 2 tables: User (12 fields) + VideoRecord (20 fields)
- One-to-Many relationship

- JSON blob for detailed comparison data
-

Slide 10: Modules Implemented

- 8 major modules:
 - User Authentication (register, login, OTP reset)
 - Video Upload & Management (async processing)
 - Single-Player Shot Analysis (full pipeline)
 - Shadow Practice Tracker (rep counting)
 - Comparison & Scoring Engine (MAE/MSE/Similarity)
 - AI Coaching Feedback (6-component)
 - Dashboard & Progress Tracking (charts)
 - Learning Center (tutorials, drills)
-

Slide 11: Implementation — Screenshots

- *(Insert 4-6 key screenshots: Landing page, Dashboard, Analysis Report, Shadow Practice)*
-

Slide 12: Testing & Results

- 3 practice videos tested → 56 total swings detected
 - Average confidence: 0.90
 - Shot distribution: Lift (53.6%), Drop (17.9%), Drive (14.3%), Smash (8.9%)
 - Quality range: 30% (poor) to 98% (excellent)
 - Manual validation against human-counted shots
-

Slide 13: Performance Metrics

- Processing: 2–5 min per 30-sec video (CPU)
 - Pose confidence threshold: 0.7
 - Classifier rejection threshold: 0.40
 - Max upload: 100 MB
 - Supported: MP4, AVI, MOV, MKV, WebM
-

Slide 14: Live Demo

- *(Show working prototype)*
 - Flow: Login → Upload video → Wait for processing → View analysis report
 - Demonstrate: Shot cards, AI coaching feedback, performance chart
-

Slide 15: Limitations & Future Work

Limitations: Camera angle dependency, CPU-only speed, rule-based classifier, single camera depth ambiguity

Future Work: Train on ShuttleSet22, mobile app, GPU acceleration, multi-player rally analysis, audio-based contact detection, wearable sensor fusion

Slide 16: Conclusion

- Successfully built a **working prototype** that analyzes badminton practice videos
 - Detects and classifies **6 shot types** with **90% average confidence**
 - Provides **measurable biomechanical feedback** using 3D joint angles
 - Generates **personalized coaching insights** to help players improve
 - **Free and accessible** — requires only a smartphone camera and a computer
-

Slide 17: References

- *(List top 5–7 references from Section 16)*
-

Slide 18: Thank You / Q&A

- **Thank You!**
 - *(Team names and contact)*
 - **Questions?**
-

18. Viva / Q&A Preparation

Q: What is the main technology behind your project?

We use MediaPipe Pose for 3D human pose estimation from a single camera video. It detects 33 body landmarks per frame. We then compute joint angles, classify shot types using a rule-based 12-phase pipeline, and generate coaching feedback by comparing the player's angles against ideal biomechanical standards.

Q: Why not use a deep learning model for shot classification?

Training a reliable deep learning classifier requires thousands of labeled examples per shot type. Available datasets like ShuttleSet have different input formats (rally-level labels, not frame-level). Our rule-based approach using body position, wrist velocity, follow-through direction, and biomechanical constraints achieves reliable results without needing a large training set. Future work could incorporate trained models when sufficient labeled data is available.

Q: How do you determine the contact frame?

We use two methods: (1) A custom-trained YOLO model detects the shuttlecock position in each frame, and we find the frame where the shuttle is closest to the wrist. (2) As a fallback, we use the wrist velocity profile — the contact frame is where wrist velocity peaks and then suddenly decelerates (acceleration spike).

Q: How accurate are your angles?

MediaPipe Pose provides estimates, not laboratory-grade measurements. The angles are approximately $\pm 5\text{-}10^\circ$ accurate depending on camera angle and lighting. We use Kalman filter smoothing (superior to simple moving averages) to reduce jitter. The system is designed for relative feedback (improvement tracking), not absolute clinical measurement.

Q: How do you handle multiple people in the video?

In single-player mode, we lock onto the largest (closest) person using bounding box size and maintain a horizontal position lock with 0.25 tolerance. We actively reject detections that appear to be the opponent (small bounding box, far from locked position, top of screen). In shadow practice mode, we use YOLOv8 with Kalman filter interpolation for robust tracking.

Q: What datasets did you use?

We cite ShuttleSet22, MultiSenseBadminton, and BFMD as related work validating our approach. For our own system, we use the player's uploaded videos as input data. The reference comparison uses static ideal biomechanical angles (scientifically benchmarked target ranges for each shot type and phase).

Q: What is the Kalman Filter and why did you use it?

A Kalman Filter is a mathematical algorithm that estimates the true state (angle) from noisy measurements by predicting future states and correcting with new observations. We use a 2D state vector [angle, angular_velocity] per joint. It's superior to simple EMA smoothing because it can predict through occlusions and handles the rapid movements of badminton wrists much better.

Q: What are the limitations of your system?

(1) Camera angle matters — side view gives best results. (2) CPU processing is slow (2-5 min per video). (3) Rule-based classification may misclassify edge cases. (4) Single camera has depth ambiguity. (5) Very dark environments can reduce detection confidence despite gamma correction.

19. Project File Structure (For Report)

```

athlete-vision/
├─ app.py                # Flask application (986 lines)
├─ models.py             # SQLAlchemy ORM models (71 lines)
├─ requirements.txt      # Python dependencies (14 packages)
├─ Procfile              # Deployment configuration
├─ Start_Athlete_Vision.bat  # Windows quick-start launcher
├─ shuttlecock_best.pt   # Custom YOLO shuttlecock model (6.3 MB)
├─ yolov8n-pose.pt       # YOLOv8 Nano-Pose model (6.8 MB)
├─ ai_engine/
│   ├── pose_analyzer.py  # 3D Pose + Phase Detection (1142 lines)
│   ├── shot_classifier.py # 12-Phase Shot Classification (867 lines)
│   ├── comparison_engine.py # MAE/MSE/Similarity Scoring (232 lines)
│   ├── feedback_generator.py # 6-Component AI Coach (389 lines)
│   ├── match_analyzer.py # Shadow Practice Tracker (699 lines)
│   ├── angle_utils.py    # 3D Angle Math + Kalman Filter (398 lines)
│   ├── shuttle_tracker.py # Smash Speed Estimation (146 lines)
│   ├── reference_builder.py # Ideal Benchmark Builder (89 lines)
│   ├── ideal_angles.py   # Static Biomechanical Angles (45 lines)
│   ├── court_mask.py     # Court Region Detection (9396 bytes)
│   ├── pose_gate.py      # Pose Quality Gate (9082 bytes)
│   └─ player_reid.py     # Player Re-identification (4829 bytes)
├─ session_coach/
│   ├── __init__.py       # Session Coaching Module
│   ├── pose_track.py     # Module initializer
│   ├── shots.py          # Pose tracking utilities
│   ├── metrics.py        # Shot detection helpers
│   └─ clips.py           # Scoring metrics
│                           # Video clip generation
├─ templates/
│   ├── base.html         # Jinja2 HTML Templates
│   ├── index.html        # Base template with navigation
│   ├── login.html        # Landing page (hero section)
│   ├── register.html     # Login form
│   ├── forgot_password.html # Registration form
│   ├── reset_password.html # Password reset request
│   ├── dashboard.html    # OTP-based password reset
│   ├── analysis.html     # Main dashboard (42 KB)
│   ├── history.html      # Analysis report page (63 KB)
│   ├── compare.html      # Session history (16 KB)
│   ├── profile.html      # Session comparison (27 KB)
│   └─ learning_center.html # User profile (7.9 KB)
│                           # Learning center (11 KB)
├─ static/
│   ├── css/              # Static Assets
│   ├── img/              # Stylesheets
│   ├── tips/             # Images
│   └─ favicon.png        # Tip images/content
│                           # App favicon
└─ uploads/               # User uploaded videos

```

```
|— processed/                # Processed outputs (clips, snapshots, chapters)
|— instance/                 # SQLite database directory
|— reference_data/           # Reference data files
|— reference_videos/         # Reference video files
|
|— docs/                     # Documentation
|   |— IMPLEMENTATION_PLAN.md # Full implementation plan
|   |— 6_DAY_AI_WORKFLOW.md  # Development workflow
|
|— evaluation_report.md      # Empirical performance evaluation
|— evaluation_results.csv    # Raw evaluation data
|— Train_Shuttlecock_Detector.ipynb # YOLO training notebook
|— Train_Shuttlecock_Detector_NEW.ipynb # Updated training notebook
```

Total Project Size: ~13 MB (excluding models and uploaded videos)

Total Lines of Code: ~5,000+ lines of Python, ~12,000+ lines of HTML/CSS/JS

Total AI Engine Modules: 12 specialized modules

20. One-Line Project Synopsis

"Athlete Vision is an AI-powered web application that analyzes badminton practice videos using 3D pose estimation, 12-phase shot classification, and biomechanical comparison to automatically detect shot types, compute joint angles, generate performance scores, and deliver personalized coaching feedback — enabling amateur players to improve their technique affordably without professional coaching fees."

Athlete Vision — AI-Powered Badminton Biomechanical Coaching System © 2026. All rights reserved.